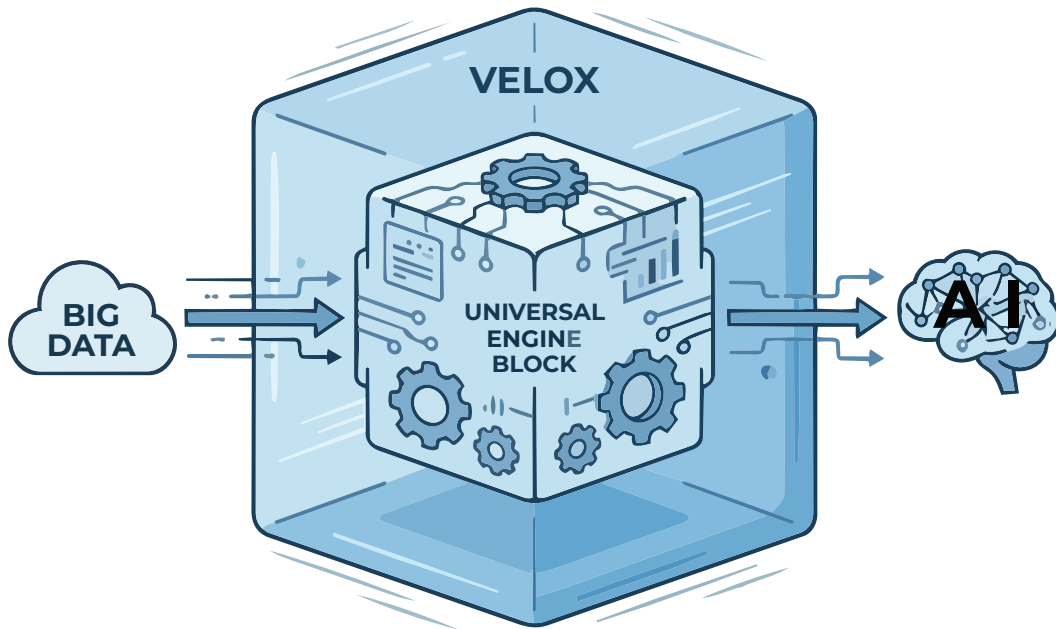


# Inside Velox, the 'Universal Engine Block' for Big Data and AI

Velox, the high-performance open source unified execution engine developed by Meta, is enabling excellent speed, consistency and efficiency in data processing.



Imagine a world where every car manufacturer had to build a completely new engine from scratch for every type of vehicle they produced -- one engine specifically designed for a nimble sports car, another for a heavy-duty pickup truck, and a completely different one for a family SUV. Historically, this is exactly how the tech industry built data management systems, creating dozens of specialised, siloed query engines tailored for specific workloads like interactive analytics, batch processing, or machine learning.

To solve this massive fragmentation and duplicated engineering effort, Meta has developed Velox: a revolutionary, open source C++ 'universal engine block' for the data ecosystem. Rather than constantly reinventing the wheel, developers can now drop Velox's highly

optimised, high-performance execution library into any system. Whether it is powering lightning-fast interactive analytics (like Presto), large-scale batch processing (like Apache Spark), or the machine learning pipelines (like PyTorch), Velox unifies the core mechanics of data processing to deliver state-of-the-art speed, unmatched consistency, and massive engineering efficiency across the entire industry.

By the early 2010s, Meta was operating one of the world's largest data platforms, processing data from billions of users while supporting thousands of engineers running analytics, A/B tests, and machine learning workloads. As data volumes grew, performance became a constant challenge.

Meta relied heavily on Presto, the distributed SQL engine it created

in 2012. Although Presto was highly efficient for a JVM-based system, the JVM itself became a bottleneck at Meta's scale. Garbage collection, JIT compilation overhead, and limited access to modern CPU features such as SIMD made it increasingly difficult to improve query performance.

## Prestissimo: The native execution experiment

To overcome these limitations, Meta launched Prestissimo (Presto C++), a project to rewrite only Presto's execution engine in native C++, while keeping the SQL parser and query planner unchanged.

During development, the team realised that the core components they were building—columnar memory structures, vectorised execution,